Large Language Model Driven Automated Network Protocol Testing

Yunze Wei*
wyz23@mails.tsinghua.edu.cn
Tsinghua University

Kaiwen Chi*
ckw24@mails.tsinghua.edu.cn
Tsinghua University

Shibo Du
dusb24@mails.tsinghua.edu.cn
Tsinghua University

Xiaohui Xie†
xiexiaohui@tsinghua.edu.cn
Tsinghua University

Ziyu Geng
gengzy22@mails.tsinghua.edu.cn
Tsinghua University

Yuwei Han
hanyw22@mails.tsinghua.edu.cn
Tsinghua University

Zhen Li
lizhen_fz@xinertel.com
Beijing Xinertel Technology Co., Ltd.

Zhanyou Li
lizy@xinertel.com
Beijing Xinertel Technology Co., Ltd.

Yong Cui†
cuiyong@tsinghua.edu.cn
Tsinghua University

Abstract

Traditional network protocol testing methods face significant challenges in adapting to rapid protocol evolution. The challenges stem primarily from protocol specification analysis and customized code development for testing. To address this, we propose *NeTestLLM*, a Large Language Model (LLM)-powered framework that automates protocol testing through two key components: (1) a hybrid test case generator that extracts protocol specifications and produces high-coverage test cases, and (2) a retrieval-feedback-enhanced engine that translates natural language descriptions into executable code. Preliminary evaluations demonstrate that *NeTestLLM* achieves 94.1% coverage on protocol specification understanding. A case study with commercial network equipment validates the practical effectiveness of our approach. Our work presents the first LLM-powered framework for automated network protocol testing to keep pace with the rapid evolution of network protocols and standards.

CCS Concepts

• **Networks** → **Protocol testing and verification**.

*Both authors contributed equally to the paper.
†Corresponding Authors: Xiaohui Xie and Yong Cui.



This work is licensed under Creative Commons Attribution International 4.0.
ANRW 25, Madrid, Spain
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2009-3/25/07
https://doi.org/10.1145/3744200.3744763

ACM Reference Format:
Yunze Wei, Kaiwen Chi, Shibo Du, Xiaohui Xie, Ziyu Geng, Yuwei Han, Zhen Li, Zhanyou Li, and Yong Cui. 2025. Large Language Model Driven Automated Network Protocol Testing. In *Applied Networking Research Workshop (ANRW 25), July 22, 2025, Madrid, Spain.* ACM, New York, NY, USA, 7 pages. https://doi.org/10.1145/3744200.3744763

1 Introduction

Modern network infrastructure relies on systematically validated protocol implementations to ensure interoperability and reliability. Network device manufacturers and purchasers must perform extensive testing to verify whether protocol implementations comply with specifications. Established organizations such as IETF, IEEE, ISO, and ITU, alongside industry alliances, continuously release and update technical standards. Conventional manual test case generation methods struggle to keep pace with the emergence of new protocol standards.

Traditional protocol testing methods rely heavily on manual effort. Engineers first need to study the protocol specification, extract key testing points, and compose test cases in natural language. Test cases are protocol-oriented but device-independent. In the second step to execute these test cases, engineers manually write test codes/configurations and deploy them to both the tester and the device under test (DUT). This process is not only labor-intensive but also error-prone and hard to ensure coverage and effectiveness. Although SCALE [7] and MESSI [19] have partially automated this process and improved test coverage, they are model-based and still require extensive manual effort in protocol modeling.

With the advancement of Large Language Models (LLMs), an increasing number of works apply LLMs in software unit testing [3, 21], penetration testing [4], and fuzz testing [12].

However, leveraging LLMs to empower network protocol testing still faces several challenges. 1) Protocol specifications are inherently complex and verbose, making it difficult for LLMs to comprehend them fully. 2) Test code is highly specialized, making automated generation by LLMs a challenge. 3) Accumulated expert knowledge is also difficult to leverage. To address these challenges, we propose *NeTestLLM*, an LLM-empowered framework that automates the process of protocol testing. Our approach includes two main components: 1) A hybrid test case generation module that extracts protocol specifications such as fields and finite state machine (FSM), and generates test cases by leveraging examples from industry and national standards. 2) A retrieval-feedback-enhanced test code generation engine that translates natural language test case descriptions into executable code.

We conducted preliminary evaluations to validate the effectiveness of *NeTestLLM*. We extracted protocol packet fields and state machine information from some classic RFCs and generated test cases based on this information, achieving 94.1% full coverage of protocol packet fields for generating both positive and negative test cases. We also conducted a comprehensive case study from RFC to executable test code and run on a testbed with commercial tester and switch.

The main contributions of this paper are as follows:

(1) The first LLM-empowered architecture specifically designed for network protocol testing.
(2) A hybrid test case generation method that combines RFC analysis with existing high-quality test cases.
(3) A retrieval-feedback-enhanced engine to generate executable test code.
(4) Preliminary system implementation and test on a testbed with commercial testers and switches.

2 Background and Challenges

2.1 Protocol Testing

Protocol testing is a rigorous process to verify whether a network device or system conforms to RFC specifications, which involves two phases: Firstly, protocol specifications are extracted through meticulous analysis of RFC documents, followed by the development of test cases that encompass core functionalities, error-handling mechanisms, and edge-case scenarios. These test cases are articulated with natural language descriptions and accompanying topologies. Such test cases are protocol-oriented and can be applied to a wide range of devices from different vendors. Secondly, these abstract test cases are translated into executable code for practical deployment. Engineers have to generate test code by composing API calls from hardware testing devices within a designated coding environment, guided by device documentation.This fundamentally constitutes repository-level code generation for a private repository.

Protocol testing is crucial to ensure interoperability among heterogeneous devices and to verify whether they meet the required performance metrics. It is applied in many scenarios: manufacturers validate protocol implementations while developing new devices or systems, and network operators conduct testing on candidate devices during procurement.

2.2 Related Work

Protocol Understanding. Some existing works utilize model-based approaches to understand protocol specifications. SAGE [24] employs a semantic parsing technique (CCG [1]) to disambiguate protocol specifications and generate code. SCALE [7] employs symbolic execution of an executable DNS model to identify RFC compliance errors in DNS nameserver implementations. MESSI [19] extends SCALE via modular exploration to generate test cases for stateful protocols such as BGP. Although these methods theoretically guarantee coverage, they require significant manual effort for modeling when generalizing to new protocols.

Recently, an increasing number of studies have explored the application of LLMs in protocol understanding. RFC-NLP [14] uses a data-driven hybrid approach to extract FSMs from RFC documents for protocol security. PROSPER [17] leverages LLMs to extract protocol specifications (such as FSM) from RFC documents. These methods provide new insights into using LLMs to understand protocol specifications.

Repository-Level Code Generation. For LLM-based code generation techniques, providing only task descriptions proves insufficient as the private repository likely wasn't included in the LLM's corpus, resulting in inherent unawareness of its comprehensive documentation. While retrieval-augmented code generation (RaCG) approaches [5, 6, 9, 11, 15, 25, 26] have attracted significant attention, these solutions predominantly operate in single-round paradigms, incapable of leveraging runtime logs to rectify non-executable generated code. PromSec [13] adopts a multi-iteration methodology that enhances code security by computing and mitigating Common Weakness Enumerations (CWEs), yet its applicability remains constrained to specific scenarios. SWE-Agent [23] enables LLMs to actively explore code repositories and interact with execution environments but lacks human-in-the-loop interfaces and demonstrates suboptimal utilization of execution results, API documentation, and code examples.

2.3 Challenges

Despite the promising capabilities of LLMs, several key challenges remain in applying them to network protocol testing.

C1: Understanding complex protocol specifications. Protocol specifications such as RFC documents are inherently complex and verbose. While LLMs excel at processing natural language, their ability to fully comprehend intricate

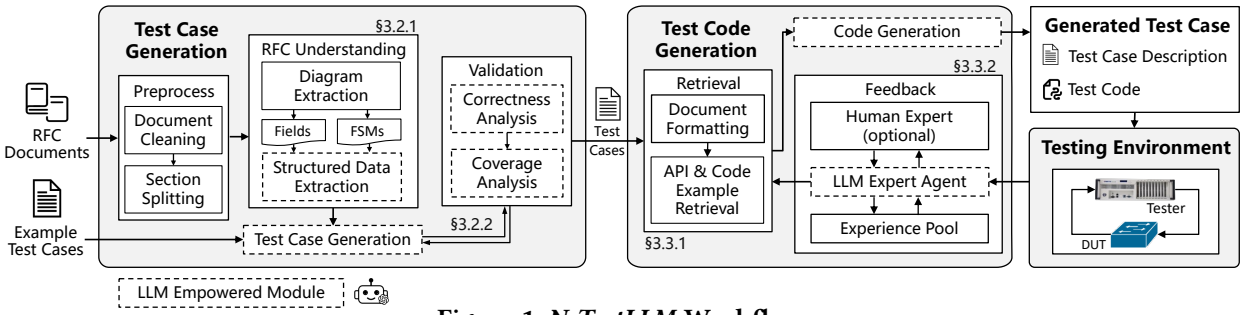


Figure 1: *NeTestLLM* Workflow

protocol fields, state machines, and edge-case behaviors remains limited. The main issue is the conflict between the ambiguous nature of LLMs and precise protocol specifications. To address this, we use structured templates to guide LLMs in extracting protocol specifications systematically.

C2: Enhancing automation in test code generation. Current RaCG approaches for private repositories generally exhibit low pass rates, making full automation challenging [22]. Furthermore, in network device testing scenarios, the correctness of generated test code cannot be inferred merely from the test outcomes observed on the DUT. Consequently, human involvement in code edition and manual review remains unavoidable. It is essential to adopt structured design to minimize repetitive human involvement, thereby enhancing the automation level of test code generation and ultimately improving its performance.

C3: Leveraging accumulated human expertise. Despite the wealth of accumulated expert knowledge in protocol testing, such as protocol understanding, existing high-quality test cases, and experience in code debugging, existing approaches fail to effectively harness this knowledge to guide LLMs. To better leverage expert knowledge, well-designed templates can be employed to enhance the protocol understanding capabilities of LLMs. Existing test cases can be used to guide the generation of high-quality test cases. Furthermore, a human-in-the-loop method can be designed to facilitate efficient code debugging by LLMs.

3 Design

3.1 *NeTestLLM* Workflow

The workflow of *NeTestLLM* (Figure 1) consists of two main components: test case generation and test code generation.

In the test case generation phase, the first step is to clean and divide the RFC document into sections. This involves using regular expressions to remove headers, footers, and other extraneous information. The table of contents is then used to extract sections as a hierarchical structure. The second step is *RFC Understanding (§ 3.2.1)*, which extracts diagram information and uses LLMs to generate structured data as test points. The final step involves *test case generation and validation*

Algorithm 1: Structured RFC data extraction

Data: RFC diagram with text G, Structured diagram data template T, Cleaned and split RFC document D
Result: Structured diagram data F
`// Step 1: Extract structured expression`
$F \leftarrow$ LLM_Extract_Structured_Data(G, T)
$R \leftarrow$ LLM_Extract_Reference_Sections(G)
foreach $r \in R$ **do**
  `// Step 2: Extract constraints`
  $c \leftarrow$ LLM_Extract_Constraints$(D[r], F)$
  $F \leftarrow F \cup c$
end
return F

(§ 3.2.2). LLMs are used to generate test cases for all possible test points in the structured data, with industry/national standard test cases as few-shot prompts. The generated test cases are validated for correctness and coverage.

In the test code generation phase, the previously generated test cases are converted into executable test code through multiple iterations. In each iteration, the first step is to *retrieve potentially relevant API interfaces and code examples (§ 3.3.1)* based on the task description and improvement suggestions from the previous round. A comprehensive prompt combines the task description with retrieved elements to generate test code using LLMs. Next, the generated code is executed and the *LLM expert agent (§ 3.3.2)* generates improvement suggestions based on the runtime log, which are then utilized for the subsequent iteration. The iterative process continues until the generated code either passes runtime validation or is approved by manual review.

3.2 Test Case Generation

3.2.1 RFC Understanding with Structured Data Extraction. RFC understanding is crucial for generating correct test cases with high coverage (C1). Inspired by recent works that leverage LLMs to facilitate protocol fuzzing [12] and protocol

specifications extraction [17], we propose a simple but effective two-step approach (Algorithm 1) to extract structured data from RFC documents. We first extend the heuristic-based RFC artifact extraction algorithm from PROSPER [17] to extract diagram-related information (i.e. ASCII art and natural language descriptions) from RFC documents, forming the input data G. Then we use an expert-designed template T as a guide for LLM to extract structural information. Only one template type is used per round to prevent confusion.

In the first step of the algorithm, we employ LLM to extract structured data representations (e.g., JSON format) from RFC diagrams presented in textual form G based on the given data structure template T. The LLM is further used to identify relevant section numbers R (references), which define the scope for extracting associated constraints. In the second step, we leverage LLM to extract constraints c related to the structured data from the previously identified sections $D[r](r \in R)$, and then integrate these constraints with the structured data F. The structured data F is validated against a manually crafted JSON schema. This process is performed iteratively until all relevant sections have been extracted.

Examples for structured data extraction. *Packet field information* is typically presented using ASCII art diagrams accompanied by field descriptions in the RFC. We define a JSON template for packet fields with the following keys: *protocol name, version number, and a list of field information.* Each field contains the *field name, field description, field length, and a list of field constraints.* In the first step of the algorithm, LLM extracts the structured field information from the given RFC text according to the template. Then, LLM is used to populate the list of constraints for each field. Finally, all constraints associated with the fields are traversed and used as the basis for test case generation. *FSM information* is extracted using a JSON template with the following keys: *set of states, events, and state transitions.* Each state transition includes the *source state, event, target state, action, and a list of constraints.* After the FSM information is extracted by LLMs, a depth-first search (DFS) algorithm is used to traverse the state transition paths, obtaining a set of paths that cover all state transitions. This set of paths serves as the basis for test case generation.

3.2.2 Test Case Generation with Validation. Test case generation is to create natural language described test cases with test steps, expected results, and corresponding test topologies. Given the demonstrated success of LLMs in software test case generation [3, 4, 21], and the inherently natural language nature of test cases, we explore the use of LLMs for this purpose. The prompt used for test case generation includes a single constraint entry from the structured data (serving as a test point), a test case template, and high-quality test cases from industry/national standards as few-shot examples. Test case examples bring together expert experience on "what makes a good test case" (C3), allowing LLMs to generate high-quality test cases consistently.

Algorithm 2: LLM expert agent in test code generation

Data: Generated code C, Running logs R, Experience pool P
Result: Improvement suggestions as feedback F

$E \leftarrow$ Parse_Error_Information(R);
// S_0, S_1', S_1, S_2, S_3 are set of suggestions.
$S_0 \leftarrow$ LLM_Suggest(C, E);
$S_1' \leftarrow$ Retrieve_Relevant_Suggestion(P, E);
$S_1 \leftarrow$ Human_Select_Suggestion_Optional($S_0 \bigcup S_1'$);
$S_2 \leftarrow$ Human_Suggest_Optional($C, E, S_0 \bigcup S_1'$);
$C' \leftarrow$ Human_Edit_Code_Optional($C, E, S_0 \bigcup S_1'$);
$S_3 \leftarrow$ LLM_Suggest_from_Difference(C, C', E);
$F \leftarrow (E, S_1 \bigcup S_2 \bigcup S_3)$; $P \leftarrow P \bigcup \{F\}$;
return F

The generated test cases are then analyzed for correctness. This process uses prompt engineering to help the LLM validate each step and extract supporting evidence from relevant RFC sections. The coverage of test cases is enhanced in two ways. Firstly, systematically traversing the structured data ensures the coverage of test points theoretically. Secondly, LLMs are used to evaluate whether the generated test cases sufficiently cover all test points (e.g., positive and negative testing) and supplement any identified gaps. This dual mechanism facilitates comprehensive coverage analysis.

3.3 Test Code Generation

3.3.1 API Documentation & Code Example Retrieval. Code generation tasks in network device testing scenarios rely on invoking API interfaces of testers and can be enhanced by API documentation and code examples [8, 22]. However, it is infeasible to input all API documentation and examples into an LLM [10, 18] due to context length limits. Consequently, retrieval of the top-K most relevant entries from API documentation and examples becomes necessary.

Following prior work, we develop a script (around 50 LoC) to extract API documents of a tester [2], employ LLMs to generate natural language summaries of code examples [10], and retrieve relevant APIs and examples using Sentence-BERT [16]. To address the granularity mismatch between high-level task descriptions and low-level API specifications, we follow the methodology of [11], fine-tuning our retrieval system using example summaries and the corresponding API descriptions to improve retrieval accuracy.

3.3.2 LLM Expert Agent-Based Feedback. As discussed in § 2.3 (C2), achieving automated test code generation remains

difficult. To address this, we propose an LLM expert agent-based feedback approach to reduce human involvement and enhance code generation automation. The proposed method consists of three components: (1) the LLM expert agent, (2) an experience pool, and (3) human expert (optional).

LLM expert agent. The operational principle of the LLM expert agent is formalized in Algorithm 2. The agent's objective is to generate natural-language improvement suggestions based on error messages in runtime logs, which serve as feedback for the next iteration of code generation.

First, by constraining the LLM to output error messages in a specified format, rule-based scripts can parse complex runtime logs to extract structured error information. Using both the generated code and error messages, the LLM directly produces initial improvement suggestions S_0.

In practice, LLM-generated code often contains common error patterns. To minimize repetitive human expert feedback, we maintain an experience pool P of historical improvement suggestions. The agent retrieves relevant solutions S_1' from this pool using error-message-based retrieval.

Human experts may then: (1) select viable solutions S_1 from S_0 and S_1', or (2) propose new solutions S_2 by referencing these suggestions. Additionally, the LLM expert agent permits human experts to directly rectify trivial errors (e.g., unmatched brackets or parameter capitalization errors). The LLM then summarizes edit differences to formalize modifications S_3. Finally, all improvement suggestions are consolidated into feedback F for the next iteration and inserted into the experience pool P.

Initial experience pool construction. The initial experience pool can be constructed from examples. Specifically, we use the functional summary of each example as the task description input, with the example code serving as the human-expert-modified code for each iteration. The LLM is then tasked with summarizing corresponding improvement suggestions, which are stored in the experience pool.

4 Evaluation

4.1 Implementation and Experiment Setup

We preliminarily implemented *NeTestLLM* in Python with 4974 lines of code. In *NeTestLLM*, we utilized LLMs in various modules, including structured data extraction, test case generation and analysis, test code generation and feedback. We employed the Qwen-2.5-Max [20] in our experiments. We use Sentence-BERT [16] for API documents & code example retrieval. We evaluated our test case understanding and generation capabilities on RFC 768 (UDP), RFC 792 (ICMP), RFC 793 (TCP), RFC 2131 (DHCP), and RFC 2328 (OSPF). We also utilized existing test cases from the National Standards and a specific power industry group as examples for our test case generation. The power industry has stringent requirements for test case quality due to its critical infrastructure nature, demanding high reliability and safety standards. Based on RFC and test case examples, we employ the *NeTestLLM* to generate test cases and test code for the tester. Each test case is also associated with several lines of configuration for the DUT, which are currently generated manually. We establish a testbed with a Xinertel DARYU series network tester and a Huawei CE6881 switch (with routing capability) as the DUT.

RFC #	FC	PC	EC	NC
RFC 768 (UDP)	4	0	0	0
RFC 792 (ICMP)	42	2	0	0
RFC 793 (TCP)	12	0	0	0
RFC 2131 (DHCP)	15	0	0	0
RFC 2328 (OSPF)	55	3	0	3

Table 1: Packet Field Understanding

4.2 Protocol Understanding

Metrics. To assess the protocol understanding capability of *NeTestLLM*, we define the following metrics to represent the coverage for specific elements of the protocol. (1) Full Coverage (FC): all the information correctly extracted; (2) Partial Coverage (PC): part of the information correctly extracted; (3) Error Coverage (EC): the extracted result is incorrect; (4) No Coverage (NC): the element is not covered.

Packet fields understanding. We evaluated *NeTestLLM*'s ability to understand protocol packet fields using five classic RFCs. We manually compared the *NeTestLLM*'s understanding results with the original field descriptions in the RFC texts to assess the correctness and coverage. The results are shown in Table 1. We can observe that most fields (94.1%) are fully covered (FC), with no fields covered with errors (EC), indicating that the LLM does not exhibit misunderstanding of the RFC. We note that a small number of fields are partially covered (PC), which is mostly due to references to external literature. Incorporating external sources is part of our future work. In addition, a few fields remain uncovered. Upon closer examination, the uncovered fields are primarily legacy duplicates maintained for backward compatibility, which the LLM ignores without compromising test case quality.

FSM understanding. We evaluated *NeTestLLM*'s ability to understand protocol FSMs with two classic RFCs: TCP and OSPF. The *NeTestLLM* successfully and accurately extracted all the states and events from the FSMs of both protocols. The extraction coverage of state transitions reached 91.9%, with errors primarily attributed to the inherent ambiguity in natural language descriptions of state machines. In OSPF, for example, expressions such as *Init or greater states* appear, making it challenging for the LLM to determine which states qualify as *greater*, thereby leading to errors.

4.3 Case Study

We conduct a case study using the protocol field testing of OSPF as an example. During the protocol understanding and test case generation phases, *NeTestLLM* extracted a total of 58 protocol fields with constraints, resulting in the generation of 146 test cases. For most protocol fields, both positive and negative test cases were produced. We open-sourced some test case generation results for community discussion[1].

We use the positive test case for the OSPF Hello Interval as an example to further illustrate the iterative feedback process in the test code generation phase.

Task description. Configure the hello intervals of the tester and DUT to 5 seconds and verify that the tester and DUT successfully establish an OSPF neighbor adjacency.

Generated Code. The generated codes are shown as follows. In the generated code, the correct API `edit_ospf()` can be retrieved, and the parameter `HelloInterval=5` can be populated according to the documentation.

```
edit_ospf(session_1, HelloInterval=5)
start_protocol(Protocols=['ospfv2'])
adjacency_established =
↪  wait_ospf_adjacency_state(session_1,
↪  State="FULL")
if adjacency_established == False: ...
```

Error information in running logs. We then executed the generated code in the test environment, which produced error *Failure('wait_ospf_adjacency_state Failure').*

Feedback. We observe that the LLM cannot directly produce correct modification proposals (S_0). This is because `Protocols=['ospfv2']` only activates the OSPF protocol without its dependent IP protocol. However, the `edit_ospf` method does not directly support the activation of the IP protocol. Therefore, it is necessary to set `Protocols=None` to enable all protocols by default, a detail not explicitly documented. Consequently, the LLM cannot recognize this critical constraint. This is a common error pattern and the LLM expert agent successfully retrieves an improvement suggestion (S_1') from the example-derived experience pool: "The `Protocols` parameter in `start_protocol()` should likely retain its default value `None`." This suggestion is subsequently incorporated by the human expert (S_1) as a feedback.

Refined code. In the regenerated code, the LLM successfully rectified the aforementioned error based on the improvement suggestion, ultimately generated correct test code.

Acceleration compared to manual methods. We continue to use the case study of OSPF packet fields to examine the two phases of test case generation and test code generation. For the test case generation phase, considering all stages including protocol understanding, test case generation and validation, the average time required to generate each test case is less than one minute. For the test code generation phase, our method achieves human-review approval in an average of 4 iterations. With *NeTestLLM* assistance, each test code generation requires 0.5 person-hour of effort on average. Compared to the baseline of 1 person-day, *NeTestLLM* significantly accelerates the test case and code generation.

[1]https://github.com/WheaterW/NetTestLLM-TestCase-Examples

5 Limitation and Future Work

Automatic understanding of protocol components. Currently, *NeTestLLM* focuses on extracting packet fields and FSMs from RFCs using manually designed templates, but RFCs also contain other structured information such as lists, algorithms, and sequence diagrams. Leveraging LLMs and multi-agent systems, it is possible to automate high-level protocol modeling and abstract functional decomposition, replacing manual analysis. Such modeling can enable the automatic generation of high-quality test cases, with mathematical tools further enhancing system capability, representing a challenging yet promising research direction.

More complete test case evaluation mechanism. The current *NeTestLLM* relies on structured data traversal and LLM self-checking to ensure test coverage and correctness, but lacks rigorous theoretical foundations. Future work can leverage high-level protocol models and mathematical tools to build a more comprehensive evaluation framework, guiding the generation of higher-quality test cases.

DUT configuration generation. While *NeTestLLM* supports automated tester script generation, DUT configuration generation remains challenging due to vendor-specific syntax and limited training data. Leveraging vendor documentation through retrieval-augmented generation or fine-tuning may improve LLM performance, but fully automated DUT configuration generation remains an open research problem.

Human-mimetic feedback for test code generation. *NeTestLLM* adopts a runtime feedback-based correction approach but currently lacks static syntax and semantic consistency checks. Case studies show that LLMs struggle to debug effectively without sufficient context, unlike human experts who explore codebases, set breakpoints, and inspect intermediate variables. While prior work explores repository interaction and log analysis, enabling LLMs to integrate both for human-like debugging remains an open challenge.

6 Conclusion

We propose *NeTestLLM*, a novel LLM-based framework for automated network protocol testing. It combines a hybrid test case generation method with a retrieval-feedback-enhanced code synthesis pipeline, enabling efficient translation from RFCs to executable test code. Preliminary experimental results and a real-world case study demonstrate the practical potential of our approach. *NeTestLLM* offers a scalable and effective solution to accelerate protocol testing.

Acknowledgment

This work is supported by the National Key R&D Program of China (No. 2023YFF0717602).

References

[1] Yoav Artzi, Nicholas FitzGerald, and Luke Zettlemoyer. 2013. Semantic Parsing with Combinatory Categorial Grammars. *ACL (Tutorial Abstracts)* 3 (2013).

[2] Huangxun Chen, Yukai Miao, Li Chen, Haifeng Sun, Hong Xu, Libin Liu, Gong Zhang, and Wei Wang. 2022. Software-defined network assimilation: bridging the last mile towards centralized network configuration management with nassim. In *Proceedings of the ACM SIGCOMM 2022 Conference*. 281–297.

[3] Yinghao Chen, Zehao Hu, Chen Zhi, Junxiao Han, Shuiguang Deng, and Jianwei Yin. 2024. Chatunitest: A framework for llm-based test generation. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*. 572–576.

[4] Gelei Deng, Yi Liu, Víctor Mayoral-Vilches, Peng Liu, Yuekang Li, Yuan Xu, Tianwei Zhang, Yang Liu, Martin Pinzger, and Stefan Rass. 2024. {PentestGPT}: Evaluating and harnessing large language models for automated penetration testing. In *33rd USENIX Security Symposium (USENIX Security 24)*. 847–864.

[5] Dawn Drain, Changran Hu, Chen Wu, Mikhail Breslav, and Neel Sundaresan. 2021. Generating code with the help of retrieved template functions and stack overflow answers. *arXiv preprint arXiv:2104.05310* (2021).

[6] Shirley Anugrah Hayati, Raphael Olivier, Pravalika Avvaru, Pengcheng Yin, Anthony Tomasic, and Graham Neubig. 2018. Retrieval-based neural code generation. *arXiv preprint arXiv:1808.10025* (2018).

[7] Siva Kesava Reddy Kakarla, Ryan Beckett, Todd Millstein, and George Varghese. 2022. {SCALE}: Automatically finding {RFC} compliance bugs in {DNS} nameservers. In *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*. 307–323.

[8] Geonmin Kim, Jaeyeon Kim, Hancheol Park, Wooksu Shin, and Tae-Ho Kim. 2024. Assessing the Answerability of Queries in Retrieval-Augmented Code Generation. *arXiv preprint arXiv:2411.05547* (2024).

[9] Yifan Li, Ensheng Shi, Dewu Zheng, Kefeng Duan, Jiachi Chen, and Yanlin Wang. 2024. RepoMinCoder: Improving Repository-Level Code Generation Based on Information Loss Screening. In *Proceedings of the 15th Asia-Pacific Symposium on Internetware*. 229–238.

[10] Dianshu Liao, Shidong Pan, Xiaoyu Sun, Xiaoxue Ren, Qing Huang, Zhenchang Xing, Huan Jin, and Qinying Li. 2024. A 3-CodGen: A Repository-Level Code Generation Framework for Code Reuse with Local-Aware, Global-Aware, and Third-Party-Library-Aware. *IEEE Transactions on Software Engineering* (2024).

[11] Zexiong Ma, Shengnan An, Bing Xie, and Zeqi Lin. 2024. Compositional API recommendation for library-oriented code generation. In *Proceedings of the 32nd IEEE/ACM International Conference on Program Comprehension*. 87–98.

[12] Ruijie Meng, Martin Mirchev, Marcel Böhme, and Abhik Roychoudhury. 2024. Large language model guided protocol fuzzing. In *Proceedings of the 31st Annual Network and Distributed System Security Symposium (NDSS)*, Vol. 2024.

[13] Mahmoud Nazzal, Issa Khalil, Abdallah Khreishah, and NhatHai Phan. 2024. PromSec: Prompt Optimization for Secure Generation of Functional Source Code with Large Language Models (LLMs). In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*. 2266–2280.

[14] Maria Leonor Pacheco, Max von Hippel, Ben Weintraub, Dan Goldwasser, and Cristina Nita-Rotaru. 2022. Automated attack synthesis by extracting finite state machines from protocol specification documents. In *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, 51–68.

[15] Md Rizwan Parvez, Wasi Uddin Ahmad, Saikat Chakraborty, Baishakhi Ray, and Kai-Wei Chang. 2021. Retrieval augmented code generation and summarization. *arXiv preprint arXiv:2108.11601* (2021).

[16] Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics. https://arxiv.org/abs/1908.10084

[17] Prakhar Sharma and Vinod Yegneswaran. 2023. Prosper: Extracting protocol specifications using large language models. In *Proceedings of the 22nd ACM Workshop on Hot Topics in Networks*. 41–47.

[18] Freda Shi, Xinyun Chen, Kanishka Misra, Nathan Scales, David Dohan, Ed H Chi, Nathanael Schärli, and Denny Zhou. 2023. Large language models can be easily distracted by irrelevant context. In *International Conference on Machine Learning*. PMLR, 31210–31227.

[19] Rathin Singha, Rajdeep Mondal, Ryan Beckett, Siva Kesava Reddy Kakarla, Todd Millstein, and George Varghese. 2024. {MESSI}: Behavioral Testing of {BGP} Implementations. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*. 1009–1023.

[20] Qwen Team. 2024. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115* (2024).

[21] Zejun Wang, Kaibo Liu, Ge Li, and Zhi Jin. 2024. HITS: High-coverage LLM-based Unit Test Generation via Method Slicing. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 1258–1268.

[22] Yixi Wu, Pengfei He, Zehao Wang, Shaowei Wang, Yuan Tian, and Tse-Hsun Chen. 2024. A Comprehensive Framework for Evaluating API-oriented Code Generation in Large Language Models. *arXiv preprint arXiv:2409.15228* (2024).

[23] John Yang, Carlos Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. Swe-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems* 37 (2024), 50528–50652.

[24] Jane Yen, Tamás Lévai, Qinyuan Ye, Xiang Ren, Ramesh Govindan, and Barath Raghavan. 2021. Semi-automated protocol disambiguation and code generation. In *Proceedings of the 2021 ACM SIGCOMM 2021 Conference*. 272–286.

[25] Daoguang Zan, Bei Chen, Yongshun Gong, Junzhi Cao, Fengji Zhang, Bingchao Wu, Bei Guan, Yilong Yin, and Yongji Wang. 2023. Private-library-oriented code generation with large language models. *arXiv preprint arXiv:2307.15370* (2023).

[26] Shuyan Zhou, Uri Alon, Frank F Xu, Zhiruo Wang, Zhengbao Jiang, and Graham Neubig. 2022. Docprompting: Generating code by retrieving the docs. *arXiv preprint arXiv: 2207.05987* (2022).